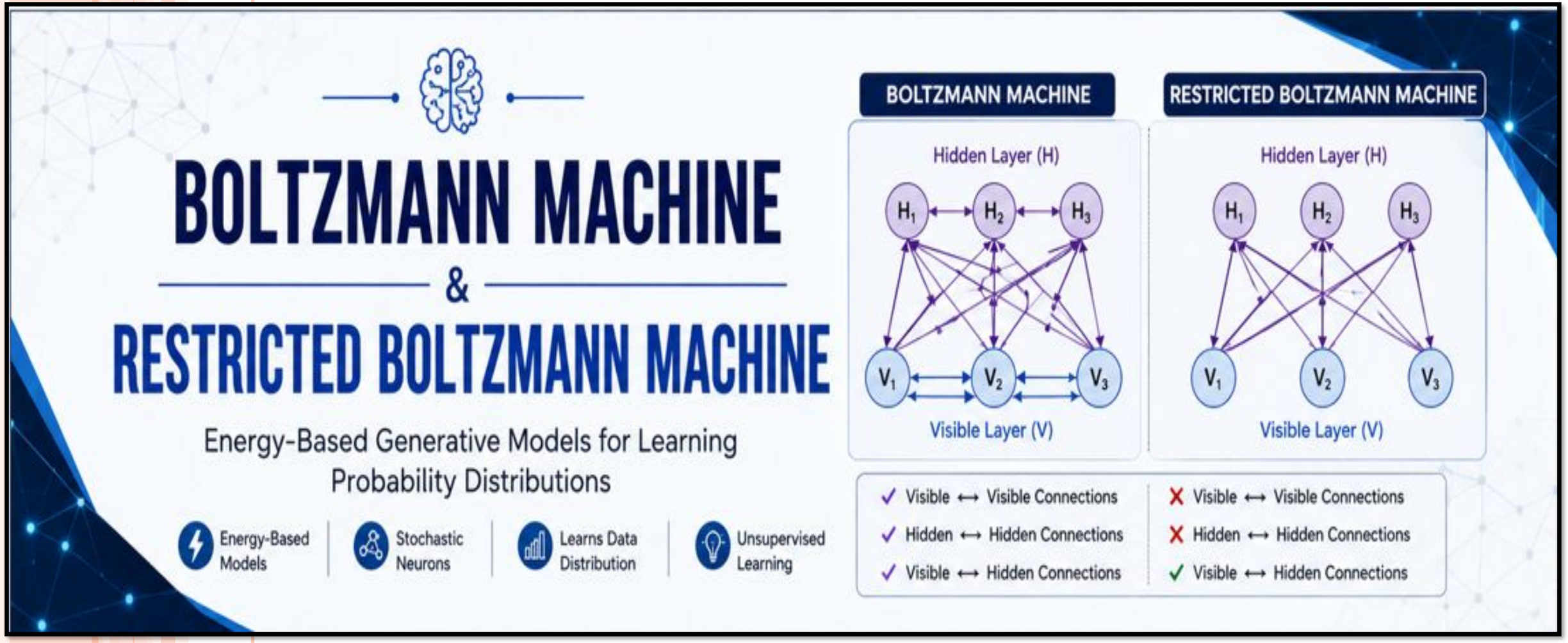
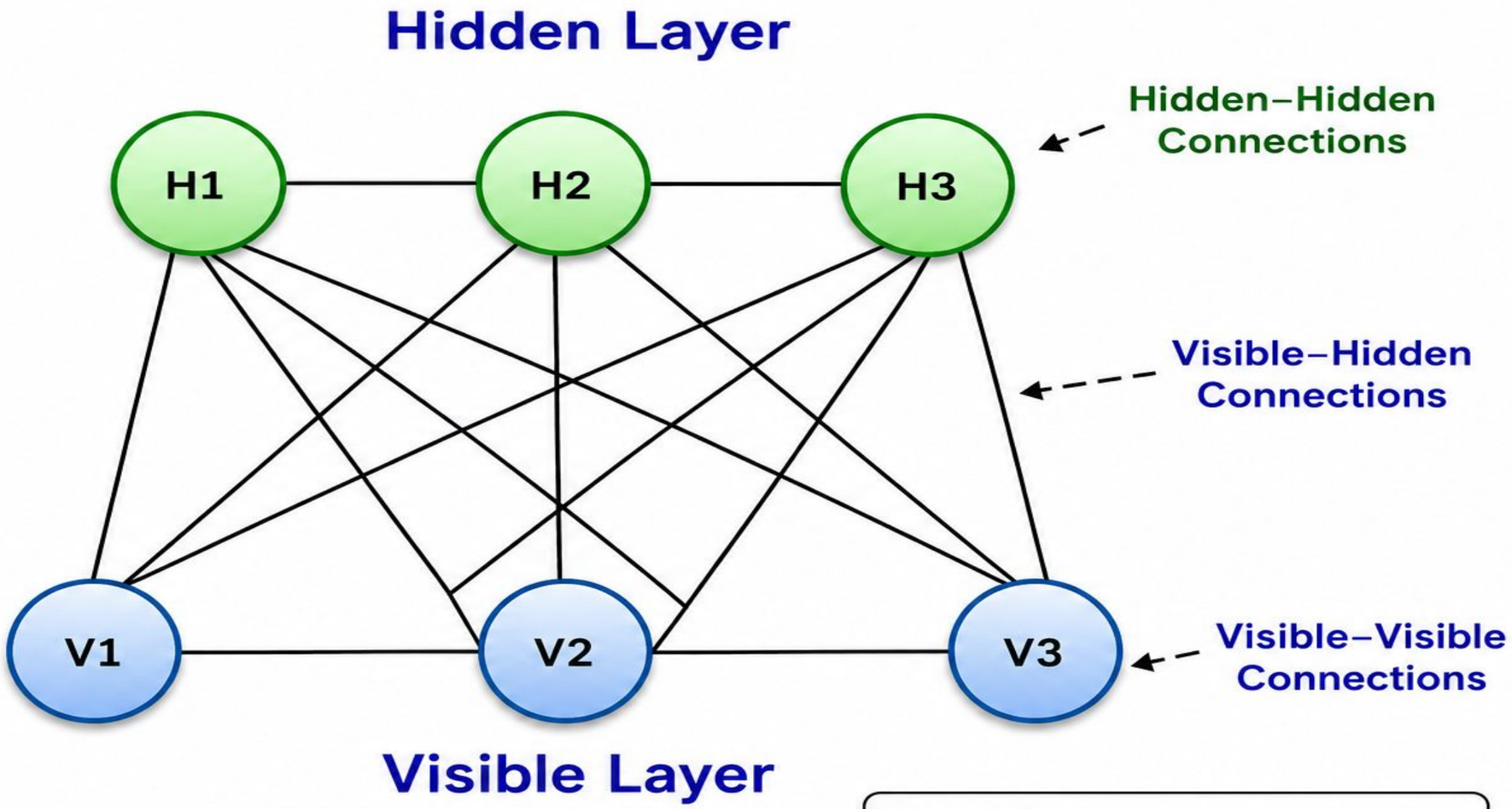
UNIT-II


BOLTZMANN MACHINE
&
RESTRICTED BOLTZMANN MACHINE
Energy-Based Generative Models for Learning Probability Distributions
Energy-Based Models
Stochastic Neurons
Learns Data Distribution
Unsupervised Learning
BOLTZMANN MACHINE
RESTRICTED BOLTZMANN MACHINE
Hidden Layer (H)
H1 H2 H3
V1 V2 V3
Visible Layer (V)
Visible ↔ Visible Connections
Hidden ↔ Hidden Connections
Visible ↔ Hidden Connections
Visible ↔ Visible Connections
Hidden ↔ Hidden Connections
Visible ↔ Hidden Connections

Boltzmann Machine

- A **Boltzmann Machine (BM)** is a **stochastic (probabilistic) neural network** introduced by **Geoffrey Hinton and Terry Sejnowski (1985)**. It **learns the probability distribution of input data** by **minimizing the energy of the network**.
- Unlike **traditional neural networks**, **neurons in a Boltzmann Machine** are **randomly activated based on probabilities instead of deterministic activation functions**.
- **Boltzmann Machine** is **an unsupervised neural network** that **learns hidden patterns** from **data**. It uses **binary neurons that turn ON or OFF based on probability and continuously adjusts its weights** to **reduce the network's energy**, **allowing it to learn the best representation of the input data.**

ARCHITECTURE OF BOLTZMANN MACHINE



Key Point:

In a Boltzmann Machine, all neurons are connected to each other.

ARCHITECTURE OF BOLTZMANN MACHINE

A Boltzmann Machine mainly contains **two types of neurons:**

1.Visible neurons

2.Hidden neurons

Both types of neurons are connected to form a network that **learns patterns from input data.**

2. **Visible Layer (V):** The **visible layer** is the part of the network that **directly receives the input data.**

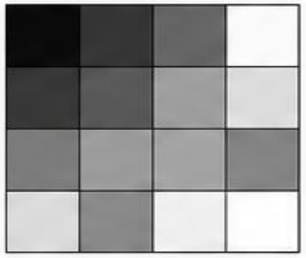
- Visible neurons are represented as: **V1,V2,V3,…,Vn**

Visible Neurons – Represents Observed Information

Visible neurons represent information that can be directly seen or measured.

Examples of Visible Neurons (Observed Information)

1. Pixel values in an image



Example Image
(4×4 pixels)



Visible Neurons (16 neurons)

Each neuron represents the intensity of one pixel (0 to 255)

2. User ratings in a recommendation system

User	Movies			
	M_1	M_2	M_3	M_4
U_1	5	3	0	4
U_2	0	4	3	2
U_3	4	0	5	0
U_4	3	2	0	5

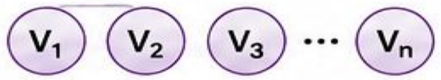


Visible Neurons (4 neurons)

Each neuron represents the rating for one movie
(0 = not watched, 1–5 = rating)

3. Words in a document

Document:
Deep learning models **learn patterns** from data.



Visible Neurons (n neurons)

Each neuron represents the presence (1) or absence (0) of a word in the vocabulary

4. Symptoms in medical data

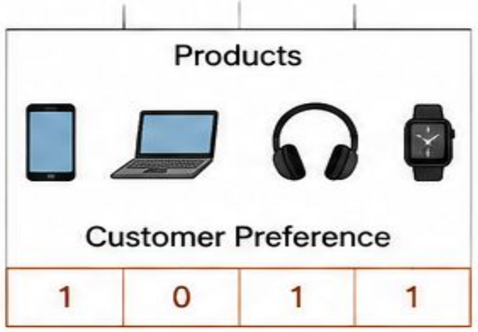
Symptoms	Present (1/0)
Fever	1
Cough	1
Headache	0
Fatigue	1
Sore Throat	0
...	...



Visible Neurons (m neurons)

Each neuron indicates whether a symptom is present (1) or not (0)

5. Product preferences of customers





Visible Neurons (4 neurons)

Each neuron represents whether the customer is interested in a product (1) or not (0)

In all these cases, the values of visible neurons are directly available in the input dataset.
Therefore, they are called **visible (observed) neurons**.

ARCHITECTURE OF BOLTZMANN MACHINE

2. Hidden Layer (H): The **hidden layer does not directly receive the original input**. Instead, it **learns important internal features** and **relationships from the visible neurons**.

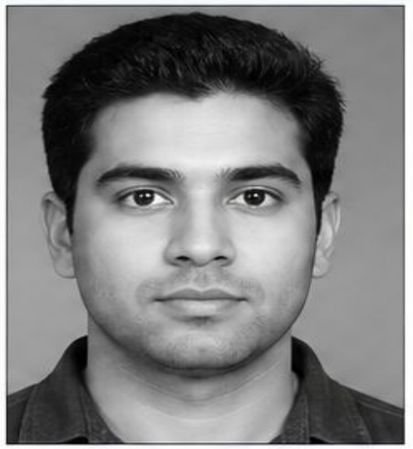
Hidden neurons are represented as: **H1,H2,H3,…,Hm**

Functions of the Hidden Layer:

1. Learns Hidden Features: The hidden layer discovers important patterns that are not directly provided in the input.

1. Hidden Layer Learns Hidden Features

The hidden layer discovers important patterns that are not directly provided in the input.



Face Image

Visible Layer: Pixels (Input)

23	45	12	67	89	34	...	23	10
34	56	78	94	120	67	...	45	22
12	34	56	78	90	45	...	32	16
...	...	...	...	...	...	...	...	...
34	67	45	23	12	34	...	67	45
12	23	34	45	56	23	...	12	10

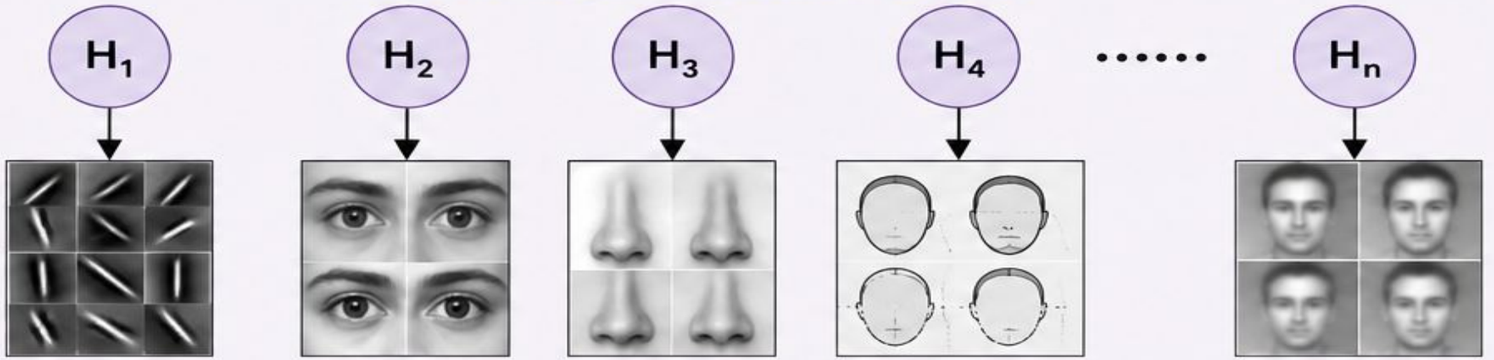
Each visible neuron represents one pixel intensity
(0 = black, 255 = white)

Visible neurons receive the raw pixel values of the image.

These values are directly available in the input data.

Hidden neurons discover and learn important patterns (features) from the input pixels.

Hidden Layer: Learned Features

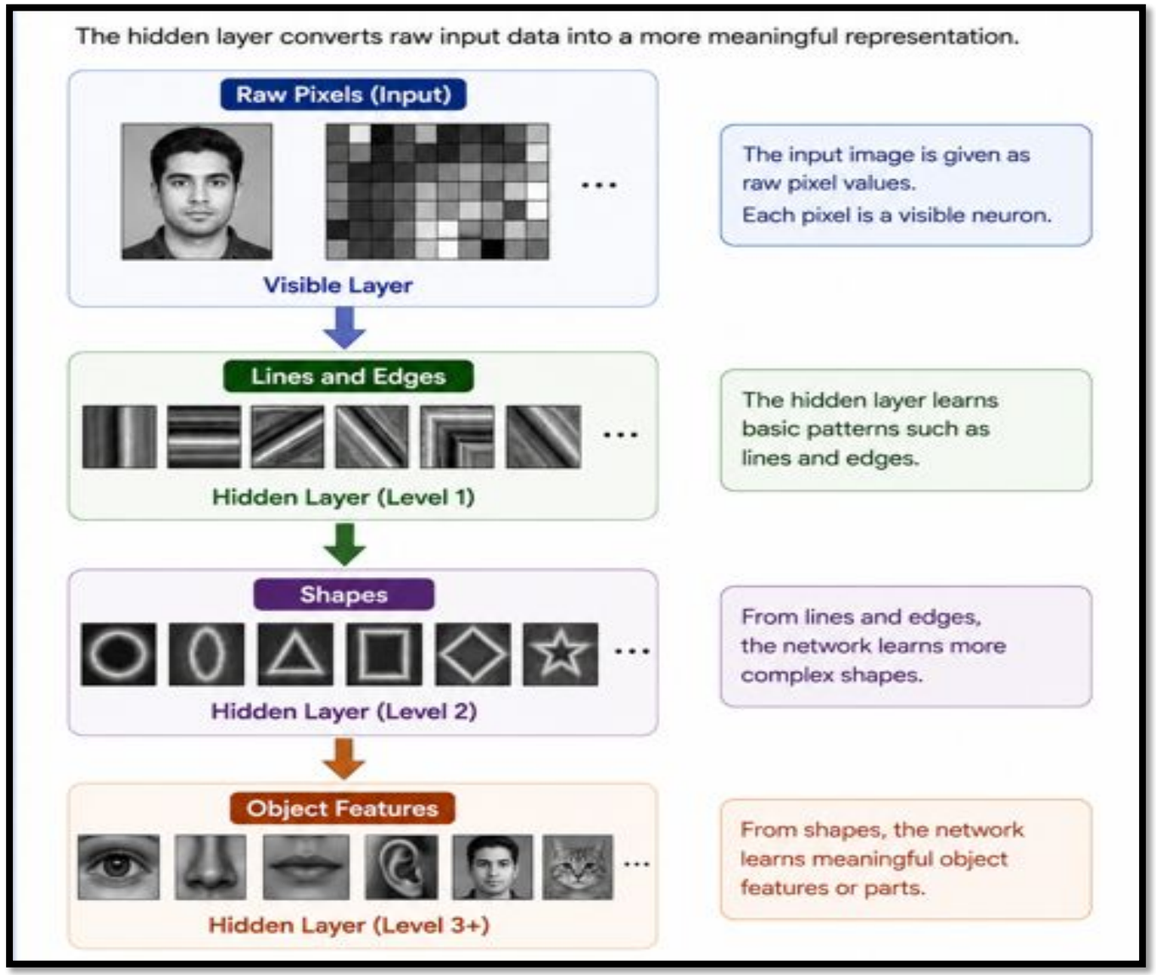


These patterns are not directly given in the input, but are automatically learned during training.

Summary: The hidden layer learns meaningful features like **edges**, **eyes**, **nose**, and **face shape** by analyzing the pixel patterns of many face images during training.

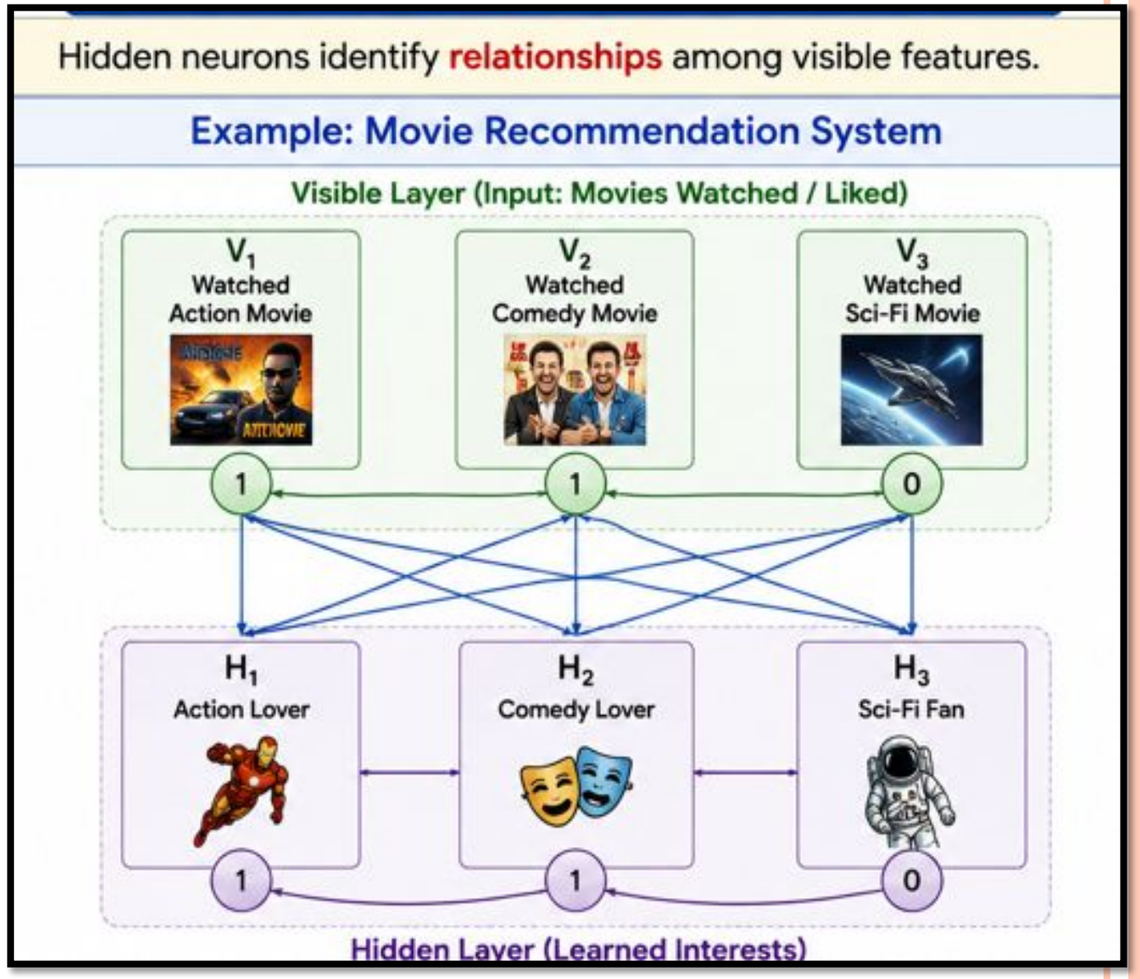
2. Creates an Internal Representation:

The hidden layer converts raw input data into a more meaningful representation.



3. Captures Relationships in Data:

Hidden neurons identify relationships among visible features.



Unique Property of Boltzmann machine

The most **important property of a Boltzmann Machine** is that **all neurons can be connected to one another**.

The network contains three main types of connections:

1. **Visible-to-visible connections**
2. **Hidden-to-hidden connections**
3. **Visible-to-hidden connections**

Unique Property of Boltzmann machine

1. **Visible ↔ Visible Connections**: **Visible neurons can communicate directly with other visible neurons. These connections help the network learn relationships between observed input features.** V1 -------- V2 -------- V3

Example

Suppose visible neurons represent symptoms:

V1 = Fever

V2 = Cough

V3 = Headache

The **visible-to-visible connection between fever and cough helps the network** understand that **these symptoms may occur together.**

Unique Property of Boltzmann machine

2. Hidden ↔ Hidden Connections: Hidden neurons are also connected to one another. H1 -------- H2 -------- H3

Example

In a face recognition system:

H1 learns eyes

H2 learns nose

H3 learns face shape

The hidden-to-hidden connections help combine these learned features to understand the complete face.

Unique Property of Boltzmann machine

3. Visible ↔ Hidden Connections: Visible neurons are connected to hidden neurons.

V1 ——— H1

V1 ——— H2

V2 ——— H1

V2 ——— H2 These connections transfer information from the input layer to the hidden layer.

Purpose

Visible-to-hidden connections help convert raw input into **meaningful hidden features.**

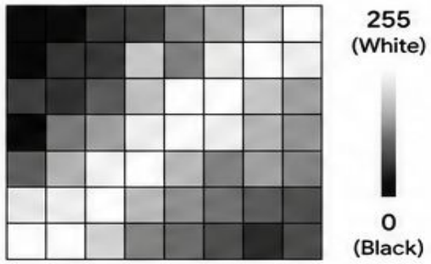
From Image Pixels to Cat Features

1. Image Pixels (Input)



Cat Image (Input)

Converted into pixel values (0 – 255)

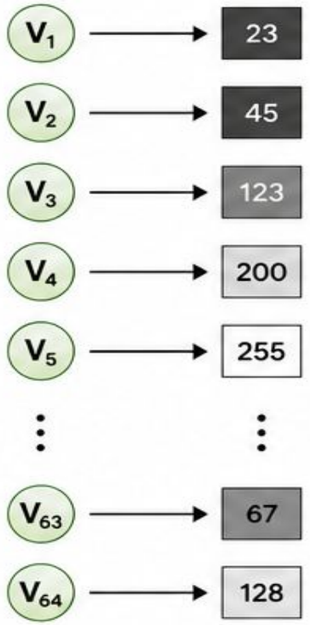


Example: 8 × 8 Pixel Grid

- The cat image is converted into pixel values (0–255).
- Each pixel value is an input value.
- These values are the raw data available to the network.

2. Visible Neurons

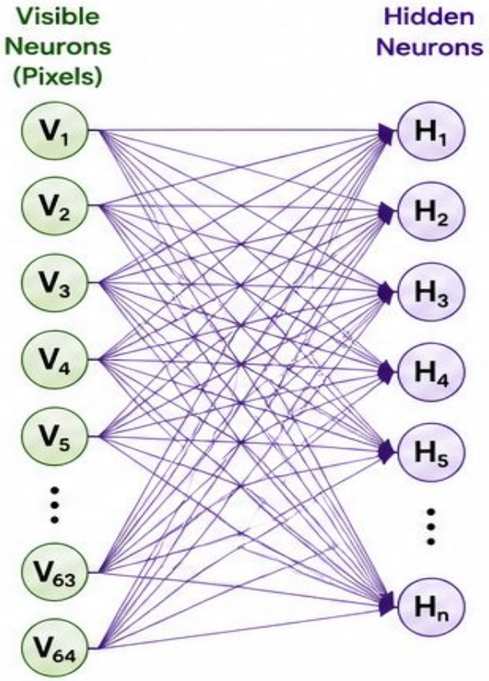
Each visible neuron represents one pixel intensity value.



- Example shows 8 × 8 image (64 pixels).
- Therefore, we have 64 visible neurons.
- Each neuron holds a pixel value (0 = black, 255 = white).

3. Visible–Hidden Connections

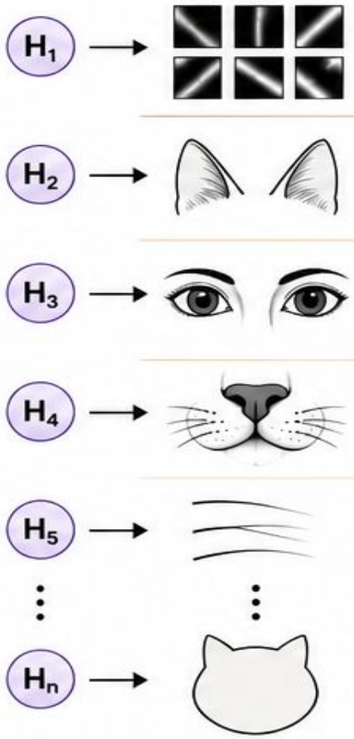
Every visible neuron connects to every hidden neuron.



- These connections carry information from pixels (visible layer) to hidden layer.
- The network learns weights for these connections during training.

4. Cat Features (Learned by Hidden Neurons)

Hidden neurons automatically learn meaningful features of the cat.



Hidden neurons combine low-level patterns (edges, parts) to form high-level features representing the cat.

Key Idea:

The network does not get edges, eyes, nose, or face shape as input.
It **discovers** these **automatically** from raw pixel values during training.

Overall Flow

Image Pixels (Raw Data) → Visible Neurons (Pixel Values) → Visible–Hidden Connections → Cat Features (Learned)

WHY IS A BOLTZMANN MACHINE FULLY CONNECTED?

- **A fully connected structure allows the model to learn many complex relationships.**

It can learn:

- **Relationships among input features**
- **Relationships among hidden features**
- **Relationships between input and hidden features**
- However, **this creates a major problem.**

Disadvantage:

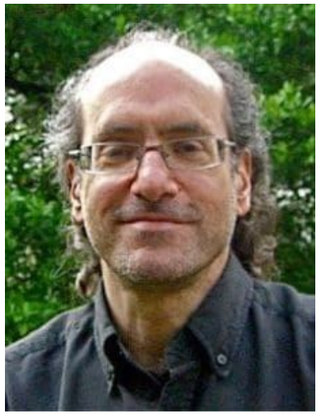
Because every neuron is connected to many other neurons:

- The **number of connections becomes very large**
- **Training becomes slow**
- **Energy calculation becomes complex**
- **More memory and computation are required**

For this reason, **the original Boltzmann Machine is difficult to train on large datasets.**

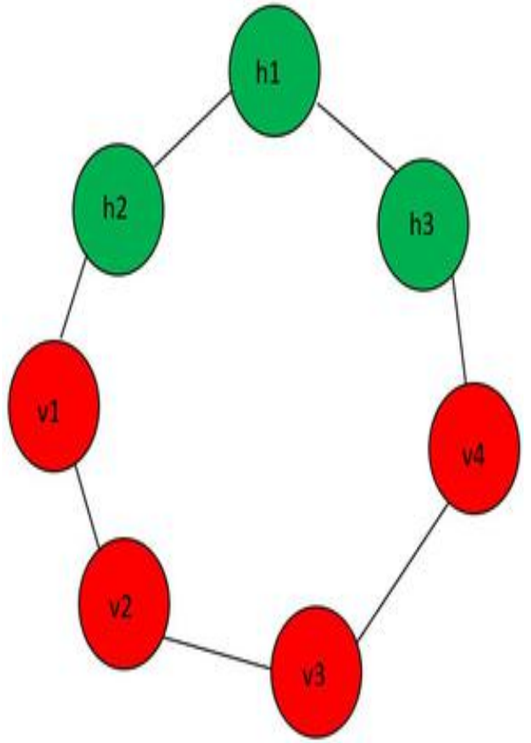
Machine

- **Restricted Boltzmann Machine** is an **undirected graphical model** that plays a major role in **Deep Learning Framework** in recent times.
- It was initially introduced as ***Harmonium*** by ***Paul Smolensky*** *in* ***1986*** and it gained big popularity in recent years in the context of the ***Netflix Prize*** ***by using Collaborative Filtering.***
- Restricted Boltzmann Machine (RBM for short) is a versatile **feature extraction method.**

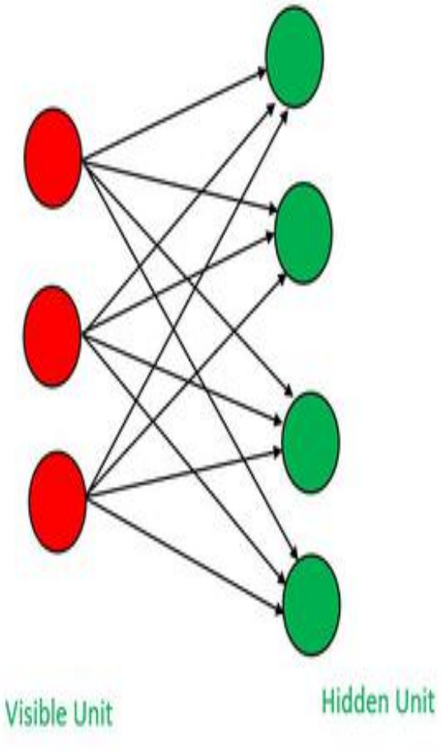


Machine

- It **is an algorithm** which is useful for
  - i) **Dimensionality Reduction**
  - ii) **Classification**
  - iii)**Regression**
  - iv) **Collaborative filtering, etc**.
- The Name suggests, The **"restricted"** part of the name "**Restricted Boltzmann Machines**" means
- **Restrict: No Visible unit is** connected to **any other Visible Unit**
- **No Hidden Unit is connected to any other Hidden Unit.**



Boltzmann Machine



Restricted Boltzmann Machine

Types of RBM

The main difference between a **Boltzmann machine** and a **restricted Boltzmann machine** is that there is **no intra layer communication**, i.e, the nodes of the **same layer are not connected** which makes them **independent from each other**.

There are **mainly two types** of **Restricted Boltzmann Machine (RBM)** based on the **types of variables** they use:

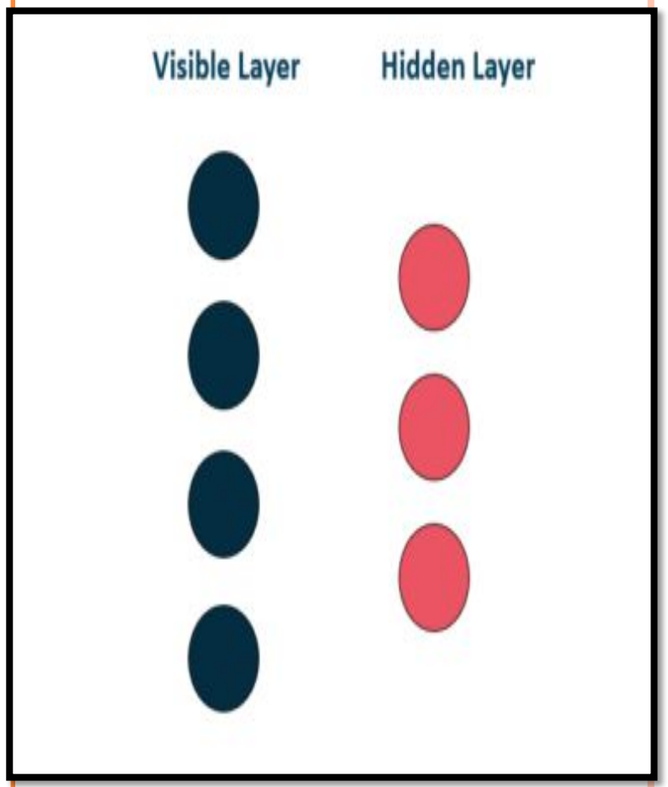
- **Binary RBM:** In a **binary RBM**, the **input and hidden units are binary variables.** Binary RBMs are often used in modeling **binary data such as images or text.**
- **Gaussian RBM:** In a **Gaussian RBM**, the **input and hidden units are continuous variables** that follow a Gaussian distribution. Gaussian RBMs are often used in **modeling continuous data** such as **audio signals or sensor data.**

WORKING PRINCIPLE OF RBM

- **Layers:** Restricted Boltzmann Machines are **shallow**, **two-layer neural nets** that constitute the building blocks of ***deep-belief networks*.**
- The **first Layer** of the RBM is called the **visible, or input layer,** and
- The **Second Layer** is the **Hidden Layer.**
- **Each circle** represents a

  Neuron-like unit called a **node.**

  The nodes are **connected to each other** **across layers,** but **no two nodes** of the **same layer are linked**.

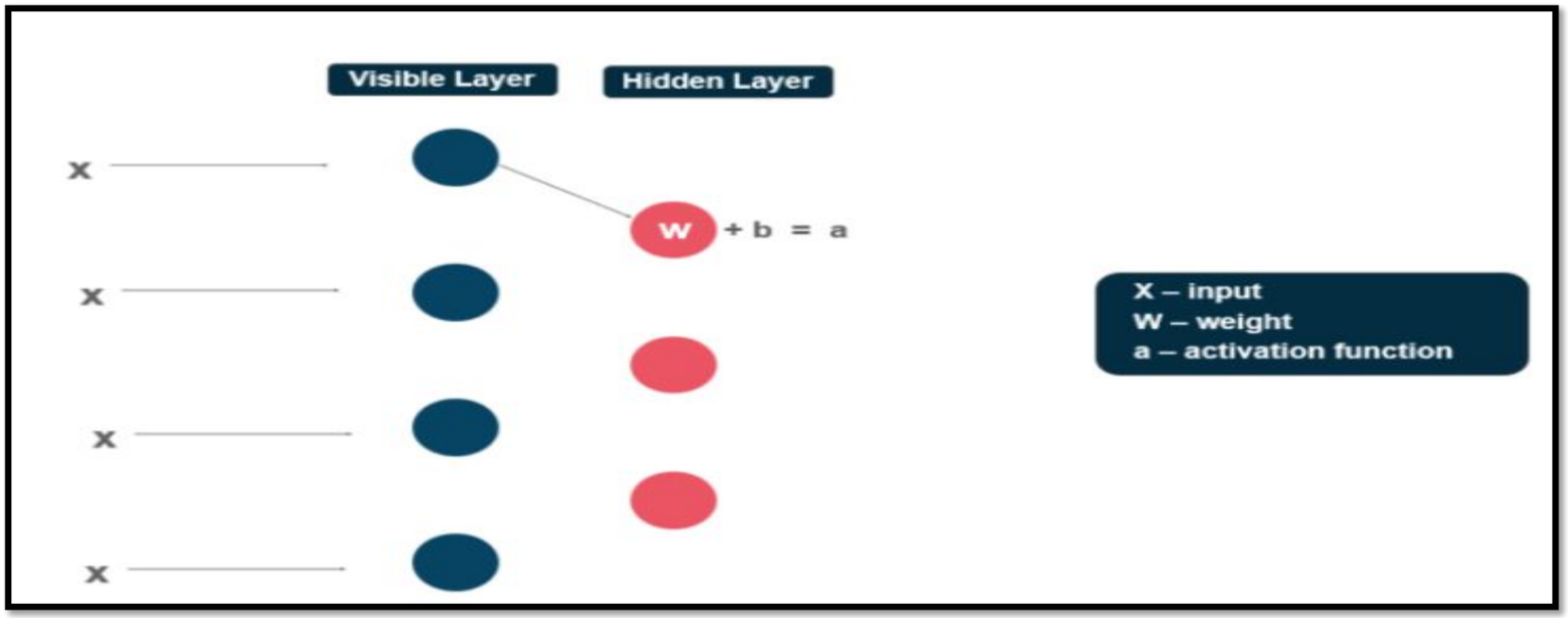


Working Principle of RBM

Each visible node takes a **low-level feature** from **an item in the dataset** to be learned.

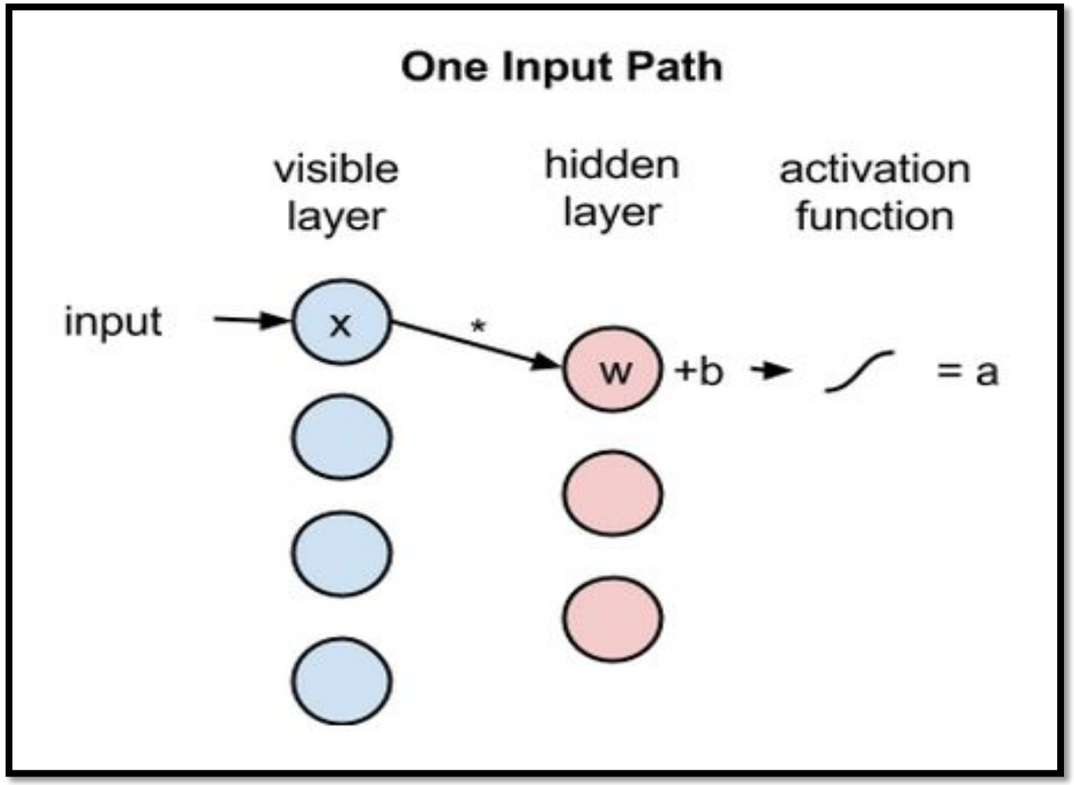
At node 1 of the hidden layer, **x is multiplied** by a ***weight*** and added to a ***bias***.

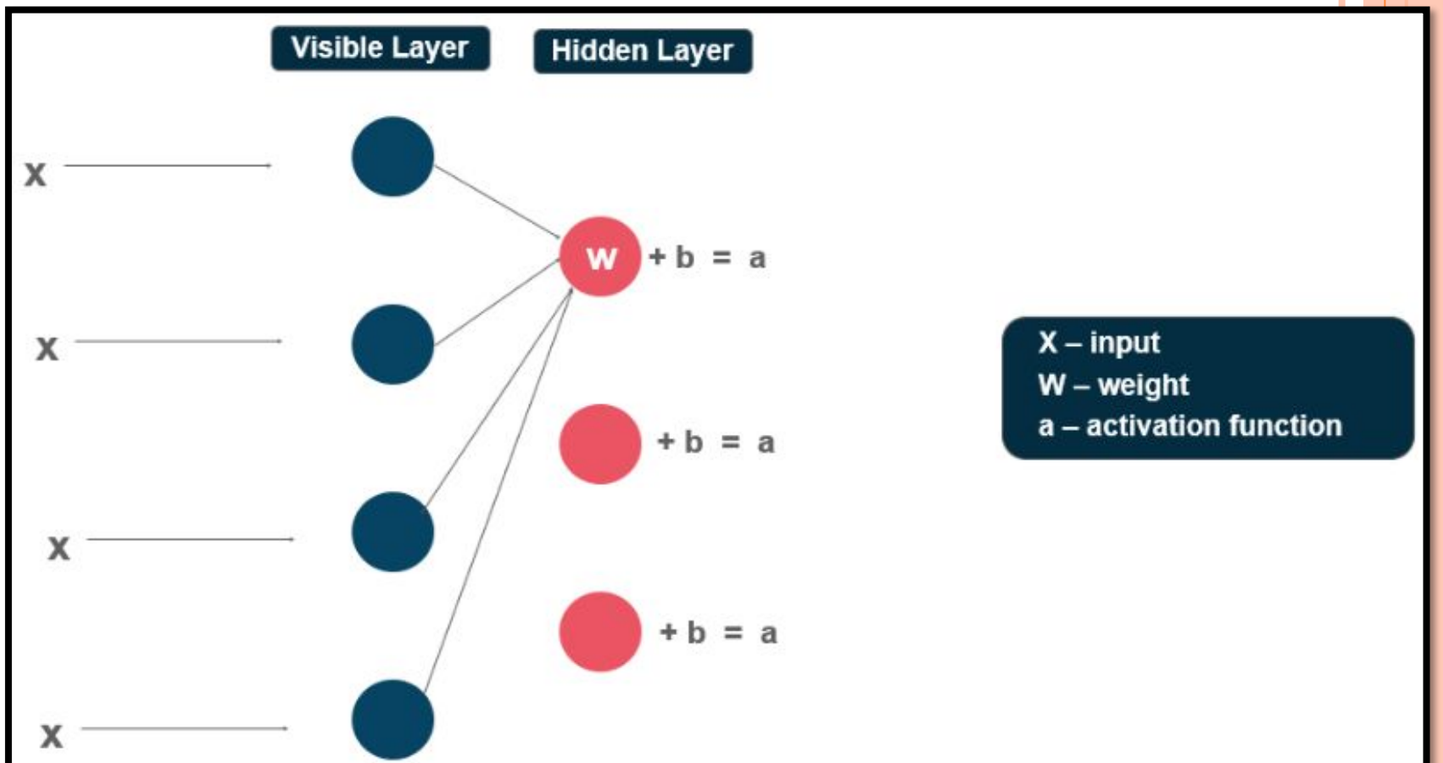
The result of those **two operations** is fed into an ***activation function***, which produces the **node's output,** or **the strength of the signal** passing through it, **given input x.**



```
activation f((weight w * input x) + bias b ) = output a
```

- Next, let's look at **how several inputs** would **combine at one hidden node.**
- Each **x is multiplied by a separate weight,** the **products are summed**, **added to a bias,** and again the result is passed through **an activation function** to **produce the node's output.**





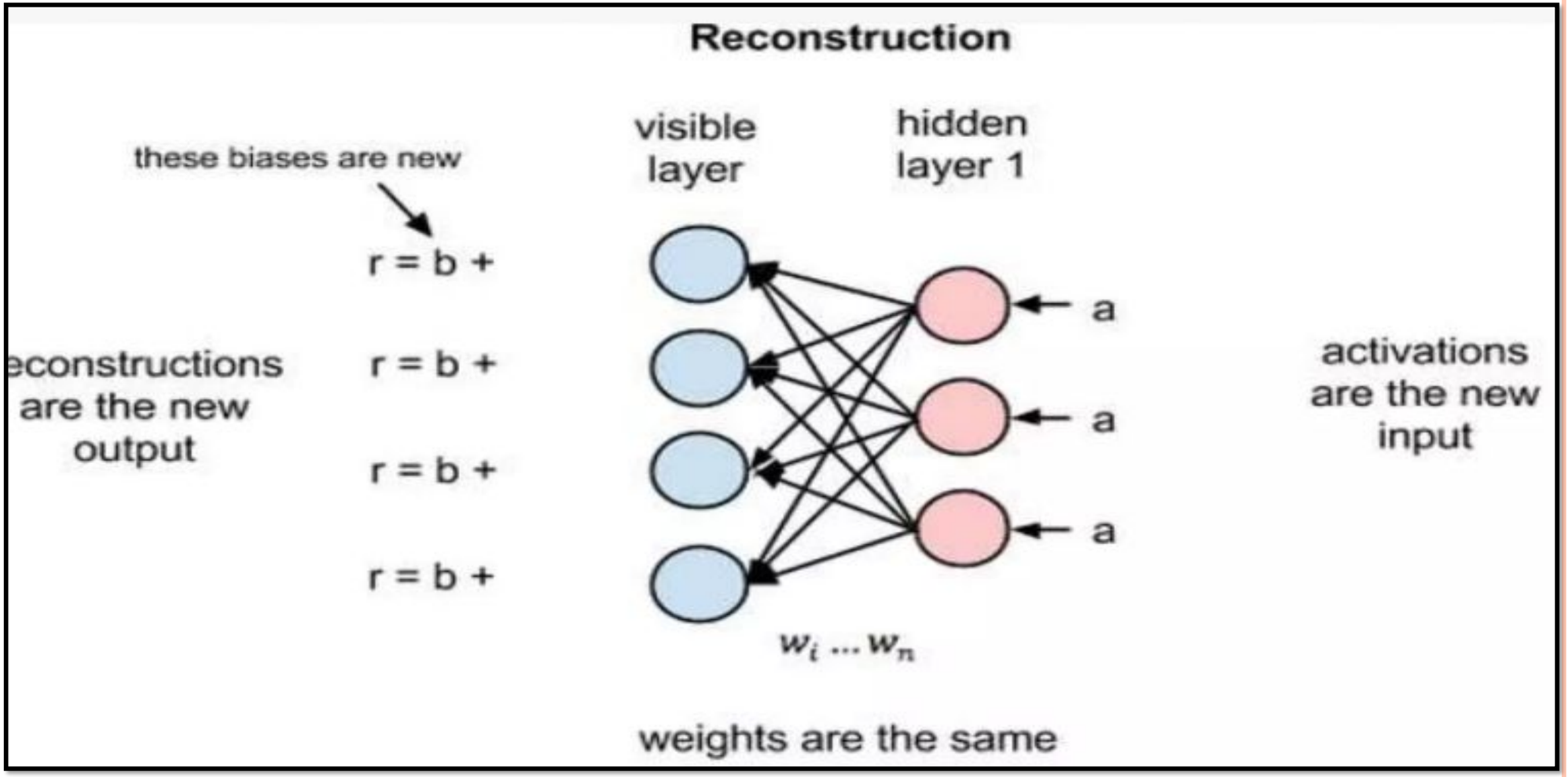
- At **each hidden node**, each **input x** is **multiplied by its respective weight w**. i.e, **a single input x** would have **three weights here**, making **12 weights altogether (4 input nodes x 3 hidden nodes).**
- The **weights between the two layers** will always ***form a matrix*** where the **rows are equal to the input nodes**, and the **columns are equal to the output nodes.**
- **Each hidden node receives the four inputs multiplied by their respective weights.**
- The **sum of those products** is **again added to a bias** (which forces at least some activations to happen), and the **result is passed through** the **activation Function** producing **one output for each hidden node**.



Reconstruction of RBM

- The Learning process consisting of **several Forward and Backward passes.**
- In the **Forward Pass,** RBM takes the **inputs and Translates them into a set of numbers** that ***encode the Inputs***
- In the **Backward Pass**, It takes the **set of numbers and translates them back** to ***form the Reconstructed Inputs.***
- In **Reconstruction**, the logic is pretty simple. You have the **activations,** which are the **inputs at this point** and then ***passed to the hidden layer and then to the input later***.
- After this, **new biases are obtained**, and **the reconstruction is the new output.**

Reconstruction of RBM



Training of Restricted Boltzmann Machine

- In RBM there are **Three Steps** through which the **entire RBM works:**
- **Step-1: FORWARD PASS:** In this phase, we take the **input layer** and using the concept of **weights and biased** we are going to **activate the hidden layer**. This process is said to be ***Feed Forward Pass***. In Feed Forward Pass we are identifying the **positive association and negative association.**

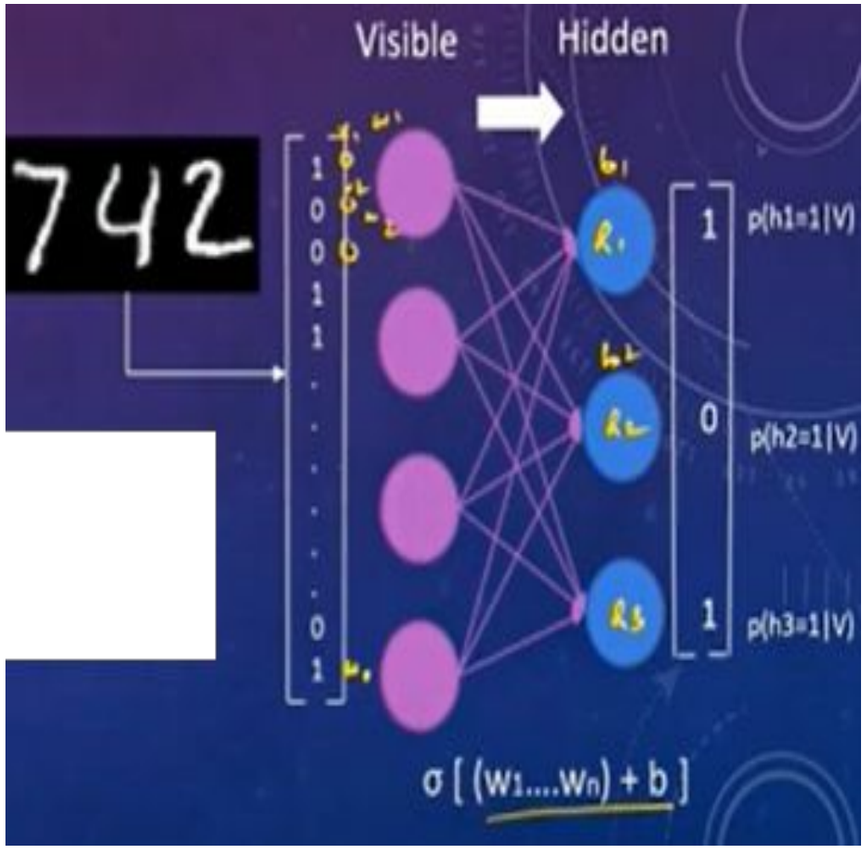
Feed Forward Equation:

Positive Association — When the association between the **visible unit and the hidden unit is positive.**

Negative Association — When the association between the **visible unit and the hidden unit is negative.**

Training of Restricted Boltzmann Machine

- **For eg:** In **Forward Pass**, The **input image is converted into Binary Values.**
- Then the **vector input** is fed into the **network**, where:
  - Its **values are multiplied by weights** and **bias is added**
  - Then the result goes through the **Activation Function, Such as Sigmoid Function,**
  - Then it represents the **Probability of the node activation.**
  - i.e, it represents the which neuron **may or may not active**



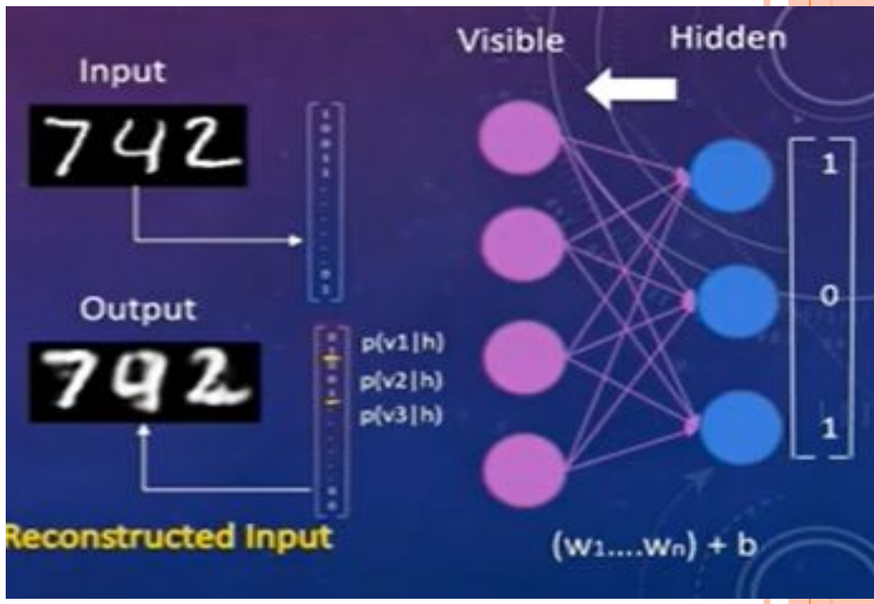
Training of Restricted Boltzmann Machine

Step-2:BACK WARD PASS: As we don't have **any output layer**. Instead of calculating the **output layer**, we are **reconstructing the input layer through the activated hidden state.**

This process is said to be **Feed Backward Pass**. We are **just backtracking the input layer through the activated hidden neurons.**

After performing this we have **reconstructed Input through the activated hidden state.** So, we can **calculate the error and adjust weight in this way:**

- **For eg:** In **Backward Pass**, The **data that is passed backward.**
- It is also combined with the same weights and bias.
- Once the **information gets to the visible Layer** , the shape of the probability distribution of the **input values, sampling the distribution, the input is reconstructed.**



Training of Restricted Boltzmann Machine

Step-3: Accessing Reconstruction Quality: Quality is compared by the original data.

The RBM calculates the error and adjust the weights and bias in order to minimize it.

Error = Reconstructed Input Layer-Actual Input layer

Adjust Weight = Input*error*learning rate (0.1)



INTERNAL TRAINING OF RESTRICTED BOLTZMANN MACHINE

- The training of the Restricted Boltzmann Machine **differs from the training of regular neural networks** via stochastic gradient descent.
- Here, **Visible Units** only talk to **Hidden Units** and
- **Hidden Units** talk to **Visible Units.**

HIDDEN

$$z_i = \sum_j w_{ji} v_i + b_i$$

$$P(h_i = 1) = \frac{1}{1 + e^{-z_i}}$$

VISIBLE

$$y_i = \sum_j w_{ji} h_i + b_i$$

$$P(v_i = 1) = \frac{1}{1 + e^{-y_i}}$$

INTERNAL TRAINING OF RESTRICTED BOLTZMANN MACHINE

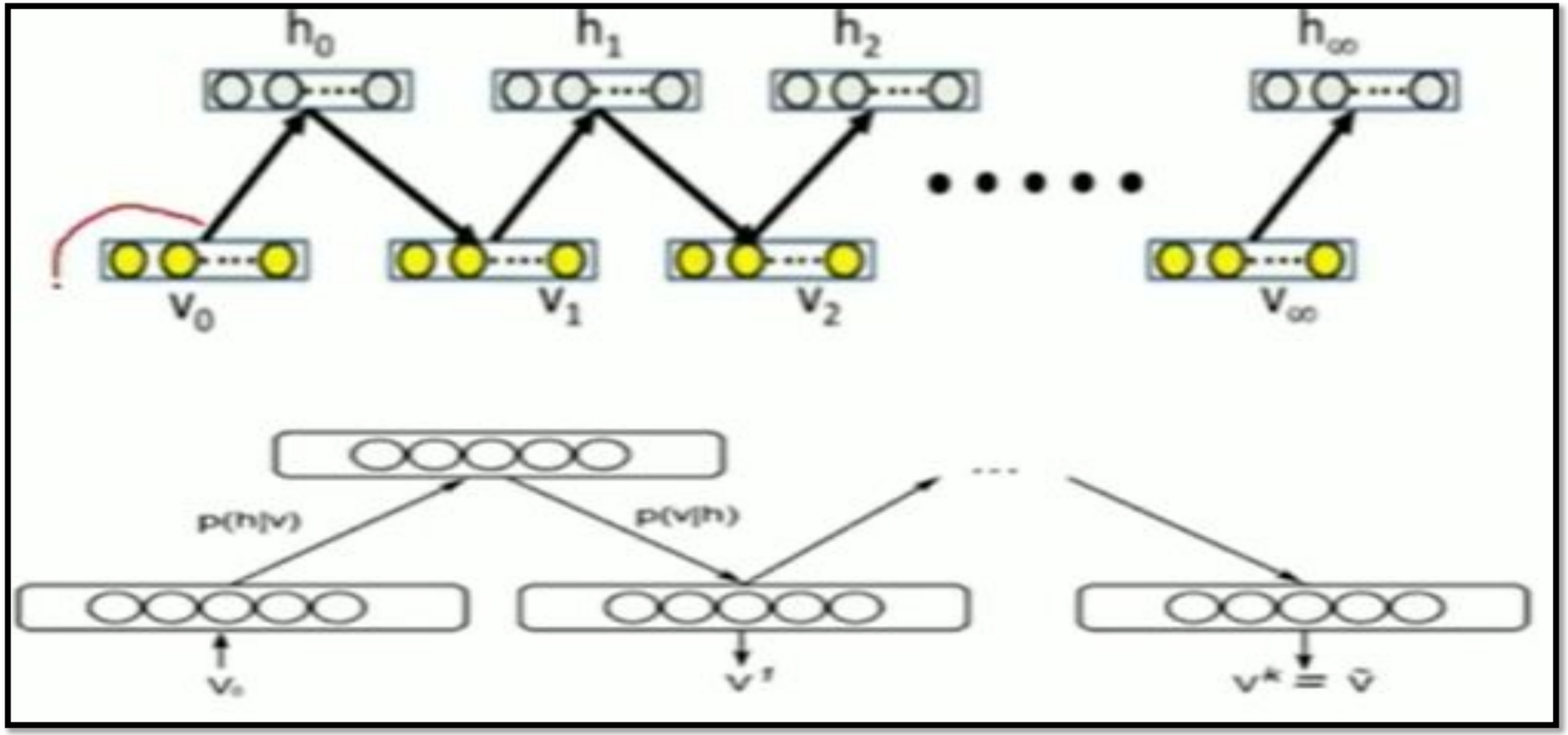
The Two main Training steps are:

1. **Gibbs Sampling:** The **first part of the training** is called **Gibbs Sampling.** Given an **input vector v** we **use p(h|v)for prediction** of the **hidden values h**.

- Remember that we all know about **P(A/B).** Ie, we are fining the **probability of A given B.**
- In the same way, P(hj=1|V) is that given the input vector V, trying to calculates the values of the Hidden Vector.

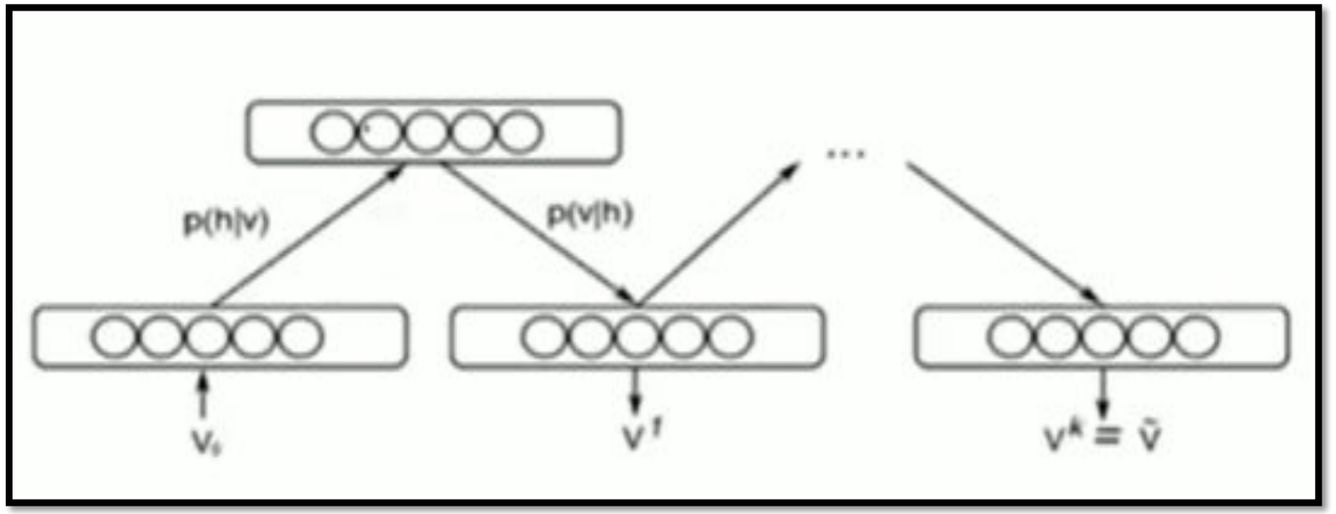
$$p(h_i = 1 \mid v) = \frac{1}{1+e(-(b_i + W_i v_i))} = \sigma\left(b_j + \sum_i v_i w_{ij}\right)$$

INTERNAL TRAINING OF RESTRICTED BOLTZMANN MACHINE



For **prediction of new input values v**. This **process is repeated k times.** After **k iterations**, we obtain **another input vector v_k** which was **recreated from original input values v_0.**

INTERNAL TRAINING OF RESTRICTED BOLTZMANN MACHINE



$$p(v_i = 1 \mid h) = \frac{1}{1+e^{(-(a_i + w_i h_j))}} = \tilde{O}\,(a_i + \sum h_j w_{ij})$$

2. **Contrastive Divergence step:** The **update of the weight matrix** happens during the **Contrastive Divergence step**. **Vectors v_0 and v_k** are used to **calculate the activation probabilities** for **hidden values h_0 and h_k :**

INTERNAL TRAINING OF RESTRICTED BOLTZMANN MACHINE

- The **difference between** the **outer products of those probabilities** with **input vectors v_0 and v_k results** in the **updated matrix :**

$$\Delta W = v_0) \otimes P(h_0 \mid v_0) - v_k \otimes P(h_k \mid v_k) - v_k)$$

- Using the **update matrix the new weights** can be **calculated with gradient ascent** given by:

$$W_{new} = W_{old} + \triangle W$$

Finally, **Contrastive divergence (CD) algorithm** is used to **train the RBM.** The algorithm performs **Gibbs sampling** and is used **inside a gradient descent procedure** (similar to the way **back propagation is used inside such a procedure** when training feed forward neural nets) to compute **weight update.**

Use cases of RBM

RBM

- **Pattern recognition :** RBM is used for **feature extraction** in **pattern recognition problems** where the challenge is to understand the **hand written text or a random pattern.**
- **Recommendation Engines** : RBM is widely used for **collaborating filtering techniques** where it is used to **predict what should be recommended to the end user.** so that the user enjoys using a **particular application or platform**.
  - **For eg : Movie Recommendation, Book Recommendation**

Hand Written Digit Recognition is a very common problem these days and is used in a variety of applications like **criminal evidence, office computerization, check verification, and data entry applications**.

It also comes with challenges like **different writing style, variations in shape and size as well as image noise**, which leads to changes in **numeral topology**.

In this a **hybrid RBM-CNN methodology** is used for **digit recognition.**

First, **features are extracted using RBM deep learning algorithms**. Then extracted features are fed to the **CNN deep learning algorithm for classification.**

RBMs are highly capable for extracting features from input data. It is designed in such a way that it can extract the features from **large and complex datasets** by introducing **hidden units in an unsupervised manner.**

